Shopping Behaviour Analysis – Data Cleaning & Exploratory Analysis

Introduction

This notebook analyses a **customer shopping behaviour dataset** to uncover meaningful insights into consumer purchasing patterns. The primary objective is to prepare a high-quality, analysis-ready dataset by identifying and resolving data inconsistencies, handling missing values, and structuring the data for business use.

This step is critical as it forms the foundation for **reliable analysis, accurate reporting, and data-driven decision-making** in later stages such as SQL analysis and Power BI dashboarding.

Objectives

- Ensure data quality and consistency
- Prepare dataset for SQL-based analysis
- Enable effective visualisation in Power BI
- Support business insights related to customer behaviour

Workflow

1. **Data Loading**
   Import the dataset and verify successful loading
2. **Initial Data Inspection**
   Understand structure, data types, and basic statistics
3. **Data Exploration (EDA)**
   Identify patterns, anomalies, and distributions
4. **Column Name Standardization**
   Clean and format column names for consistency
5. **Feature Engineering**
   Create new meaningful features (e.g., segments, categories)
6. **Handling Missing Values**
   Detect and appropriately treat null or inconsistent data
7. **Removing Redundant Columns**
   Drop irrelevant or duplicate columns to streamline the dataset
8. **Data Export & Integration**
   Save the cleaned dataset and prepare for SQL and Power BI usage

---

Outcome

At the end of this process, the dataset becomes:

- Clean and structured
- Consistent and reliable
- Ready for advanced analysis (SQL + Power BI)

Next Steps

- Perform **SQL-based business analysis**
- Build an **interactive Power BI dashboard**
- Generate actionable business insights

In [ ]:

In [ ]:

Data Loading

In [1]:
```
import pandas as pd

# Load dataset
df = pd.read_csv(r"C:\Users\Prashanth\Documents\customer-trends-data-analysis-SQL-Python-PowerBI\Dataset\customer_shopping_behavior.csv")
df.head()     # Preview the first 5 rows
```

Out[1]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	PayPal	Annually

In [ ]:

In [ ]:

Initial Data Inspection

In [2]:
```
# Dataset shape
print(f"Total rows : {df.shape[0]}")
print(f"Total columns : {df.shape[1]}")
```

```
Total rows : 3900
Total columns : 18
```

In [3]:
```
# Dataset structure
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

In [ ]:

In [4]:
```
# Summary statistics for numeric columns
df.describe(exclude="object").T
```

Out[4]:

	count	mean	std	min	25%	50%	75%	max
Customer ID	3900.0	1950.500000	1125.977353	1.0	975.75	1950.5	2925.25	3900.0
Age	3900.0	44.068462	15.207589	18.0	31.00	44.0	57.00	70.0
Purchase Amount (USD)	3900.0	59.764359	23.685392	20.0	39.00	60.0	81.00	100.0
Review Rating	3863.0	3.750065	0.716983	2.5	3.10	3.8	4.40	5.0
Previous Purchases	3900.0	25.351538	14.447125	1.0	13.00	25.0	38.00	50.0

In [ ]:

In [5]:
```
# Summary statistics for text columns
df.describe(include="object").T
```

Out[5]:

	count	unique	top	freq
Gender	3900	2	Male	2652
Item Purchased	3900	25	Blouse	171
Category	3900	4	Clothing	1737
Location	3900	50	Montana	96
Size	3900	4	M	1755
Color	3900	25	Olive	177
Season	3900	4	Spring	999
Subscription Status	3900	2	No	2847
Shipping Type	3900	6	Free Shipping	675
Discount Applied	3900	2	No	2223
Promo Code Used	3900	2	No	2223
Payment Method	3900	6	PayPal	677
Frequency of Purchases	3900	7	Every 3 Months	584

In [ ]:

In [ ]:

Data Exploration

In [6]:
```
# Checking column names
df.columns
```

Out[6]:
```
Index(['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category',
       'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season',
       'Review Rating', 'Subscription Status', 'Shipping Type',
       'Discount Applied', 'Promo Code Used', 'Previous Purchases',
       'Payment Method', 'Frequency of Purchases'],
      dtype='object')
```

In [7]:
```
# Inspect unique values
for i in df.columns:
    print(f"\nColumn Name: {i}")   # Column Name
    print(f"Data Type: {df[i].dtype}")  # Data Type
    print(f"Unique Count: {df[i].nunique()}") # Unique Count
    print(df[i].unique()) # Unique Values
    print("-" * 50)
```

```
Column Name: Customer ID
Data Type: int64
Unique Count: 3900
[   1    2    3 ... 3898 3899 3900]
--------------------------------------------------

Column Name: Age
Data Type: int64
Unique Count: 53
[55 19 50 21 45 46 63 27 26 57 53 30 61 65 64 25 52 66 31 56 18 38 54 33
 36 35 29 70 69 67 20 39 42 68 49 59 47 40 41 48 22 24 44 37 58 32 62 51
 28 43 34 23 60]
--------------------------------------------------

Column Name: Gender
Data Type: object
Unique Count: 2
['Male' 'Female']
--------------------------------------------------

Column Name: Item Purchased
Data Type: object
Unique Count: 25
['Blouse' 'Sweater' 'Jeans' 'Sandals' 'Sneakers' 'Shirt' 'Shorts' 'Coat'
 'Handbag' 'Shoes' 'Dress' 'Skirt' 'Sunglasses' 'Pants' 'Jacket' 'Hoodie'
 'Jewelry' 'T-shirt' 'Scarf' 'Hat' 'Socks' 'Backpack' 'Belt' 'Boots'
 'Gloves']
--------------------------------------------------

Column Name: Category
Data Type: object
Unique Count: 4
['Clothing' 'Footwear' 'Outerwear' 'Accessories']
--------------------------------------------------

Column Name: Purchase Amount (USD)
Data Type: int64
Unique Count: 81
[ 53  64  73  90  49  20  85  34  97  31  68  72  51  81  36  38  48  62
  37  88  22  25  56  94  79  67  91  33  69  45  60  76  39 100  21  43
  54  55  30  28  59  24  83  46  70  29  26  58  96  40  32  95  41  47
  99  87  92  98  89  35  75  65  86  84  77  27  23  74  80  42  61  82
  78  93  50  57  66  44  52  71  63]
--------------------------------------------------

Column Name: Location
Data Type: object
Unique Count: 50
['Kentucky' 'Maine' 'Massachusetts' 'Rhode Island' 'Oregon' 'Wyoming'
 'Montana' 'Louisiana' 'West Virginia' 'Missouri' 'Arkansas' 'Hawaii'
 'Delaware' 'New Hampshire' 'New York' 'Alabama' 'Mississippi'
 'North Carolina' 'California' 'Oklahoma' 'Florida' 'Texas' 'Nevada'
 'Kansas' 'Colorado' 'North Dakota' 'Illinois' 'Indiana' 'Arizona'
 'Alaska' 'Tennessee' 'Ohio' 'New Jersey' 'Maryland' 'Vermont'
 'New Mexico' 'South Carolina' 'Idaho' 'Pennsylvania' 'Connecticut' 'Utah'
 'Virginia' 'Georgia' 'Nebraska' 'Iowa' 'South Dakota' 'Minnesota'
 'Washington' 'Wisconsin' 'Michigan']
--------------------------------------------------

Column Name: Size
Data Type: object
Unique Count: 4
['L' 'S' 'M' 'XL']
--------------------------------------------------

Column Name: Color
Data Type: object
Unique Count: 25
['Gray' 'Maroon' 'Turquoise' 'White' 'Charcoal' 'Silver' 'Pink' 'Purple'
 'Olive' 'Gold' 'Violet' 'Teal' 'Lavender' 'Black' 'Green' 'Peach' 'Red'
 'Cyan' 'Brown' 'Beige' 'Orange' 'Indigo' 'Yellow' 'Magenta' 'Blue']
--------------------------------------------------

Column Name: Season
Data Type: object
Unique Count: 4
['Winter' 'Spring' 'Summer' 'Fall']
```

```
--------------------------------------------------

Column Name: Review Rating
Data Type: float64
Unique Count: 26
[3.1 3.5 2.7 2.9 3.2 2.6 4.8 4.1 4.9 4.5 4.7 2.8 4.6 3.3 4.4 3.6 5.  4.
 nan 4.2 3.7 3.9 3.  3.8 3.4 4.3 2.5]
--------------------------------------------------

Column Name: Subscription Status
Data Type: object
Unique Count: 2
['Yes' 'No']
--------------------------------------------------

Column Name: Shipping Type
Data Type: object
Unique Count: 6
['Express' 'Free Shipping' 'Next Day Air' 'Standard' '2-Day Shipping'
 'Store Pickup']
--------------------------------------------------

Column Name: Discount Applied
Data Type: object
Unique Count: 2
['Yes' 'No']
--------------------------------------------------

Column Name: Promo Code Used
Data Type: object
Unique Count: 2
['Yes' 'No']
--------------------------------------------------

Column Name: Previous Purchases
Data Type: int64
Unique Count: 50
[14  2 23 49 31 19  8  4 26 10 37 34 44 36 17 46 50 22 32 40 16 13  7 41
 45 38 48 18 15 25 39 35 29 21 43  3  5 24 42 47 28 20 33  1  9 12 27 11
 30  6]
--------------------------------------------------

Column Name: Payment Method
Data Type: object
Unique Count: 6
['Venmo' 'Cash' 'Credit Card' 'PayPal' 'Bank Transfer' 'Debit Card']
--------------------------------------------------

Column Name: Frequency of Purchases
Data Type: object
Unique Count: 7
['Fortnightly' 'Weekly' 'Annually' 'Quarterly' 'Bi-Weekly' 'Monthly'
 'Every 3 Months']
--------------------------------------------------
```

In [ ]:

In [ ]:

Column Name Cleaning

In [8]:

```python
# Standardizing column names
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})

# Verify changes
df.columns
```

Out[8]:

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

In [ ]:

In [ ]:

Feature Engineering

In [9]:
```
# Creating age groups
df['age_group'] = pd.cut(
    df['age'],
    bins=[0,19,35,55,100],
    labels=['Teen','Young','Middle Age','Senior']
)

df[['age','age_group']].head()
```

Out[9]:

	age	age_group
0	55	Middle Age
1	19	Teen
2	50	Middle Age
3	21	Young
4	45	Middle Age

In [ ]:

In [ ]:

Handling Missing Values

In [10]:
```
# Checking missing values
df.isnull().sum()[df.isnull().sum() > 0]
```

Out[10]:
```
review_rating    37
dtype: int64
```

In [11]:
```
# Fill missing review ratings using category median
category_median = df.groupby('category')['review_rating'].median()
df['review_rating'] = df['review_rating'].fillna(df['category'].map(category_median))
```

In [12]:
```
# Checking for remaining missing values after imputation
df.isnull().sum()[df.isnull().sum() > 0]
```

Out[12]:
```
Series([], dtype: int64)
```

In [ ]:

In [ ]:

Removing Redundant Columns

In [13]:
```
# Comparing both columns
df[["discount_applied", "promo_code_used"]].head(5)
```

Out[13]:

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes

In [14]:
```
# Check if promo_code_used and discount_applied are identical
(df['promo_code_used'] == df['discount_applied']).all()
```

Out[14]:
```
np.True_
```

In [16]:
```
# Drop redundant column
df = df.drop(columns=['promo_code_used','frequency_of_purchases'])

# Verify
df.columns
```

Out[16]:
```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'age_group'],
      dtype='object')
```

In [17]:
```
df.head()
```

Out[17]:

	customer_id	age	gender	item_purchased	category	purchase_amount	location	size	color	season	review_rating	subscription_status	shipping_type	discount_applied	previous_purchases	payment_method	age_group
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	14	Venmo	Middle Age
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	2	Cash	Teen
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	23	Credit Card	Middle Age
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	49	PayPal	Young
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	31	PayPal	Middle Age

In [ ]:

In [ ]:

Saving Cleaned Data to CSV file

In [24]:
```
# Save cleaned dataset in the same folder as original dataset
df.to_csv(
    r"C:\Users\Prashanth\Documents\customer-trends-data-analysis-SQL-Python-PowerBI\Dataset\cleaned_customer_shopping_behavior.csv",
    index=False
)
```

In [ ]:

In [ ]:

Loading Data into SQL SERVER

In [18]:
```
!pip install pyodbc sqlalchemy
```

```
Requirement already satisfied: pyodbc in c:\users\prashanth\anaconda3\lib\site-packages (5.2.0)
Requirement already satisfied: sqlalchemy in c:\users\prashanth\anaconda3\lib\site-packages (2.0.39)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\prashanth\anaconda3\lib\site-packages (from sqlalchemy) (3.1.1)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\prashanth\anaconda3\lib\site-packages (from sqlalchemy) (4.15.0)
```

In [ ]:

In [19]:
```
from sqlalchemy import create_engine
from urllib.parse import quote_plus

driver = quote_plus("ODBC Driver 17 for SQL Server")

engine = create_engine(
    f"mssql+pyodbc://@Prashanth\\SQL_2025_NIT/customer_shopping_behavior?driver={driver}&trusted_connection=yes"
)
```

In [20]:
```
engine.connect()
```

Out[20]:
```
<sqlalchemy.engine.base.Connection at 0x295369357f0>
```

In [21]:
```
df.to_sql("customer", engine, if_exists="replace", index=False, chunksize=1000)
```

Out[21]: 87

In [22]:
```
# Read back sample (SQL Server uses TOP instead of LIMIT)
pd.read_sql("SELECT TOP 5 * FROM customer;", engine)
```

Out[22]:

	customer_id	age	gender	item_purchased	category	purchase_amount	location	size	color	season	review_rating	subscription_status	shipping_type	discount_applied	previous_purchases	payment_method	age_group
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	14	Venmo	Middle Age
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	2	Cash	Teen
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	23	Credit Card	Middle Age
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	49	PayPal	Young
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	31	PayPal	Middle Age

In [ ]:

In [ ]: